

BBT RADAR — Failure Prediction, Environment Compare & Dev-PR Correlation

Architecture Design Document

> **Status:** Draft v1.0

> **Scope:** Three new capabilities layered onto the existing BBT RADAR (BBT QualityIQ) dashboard

> **Author:** Chief Architect (derived from MCQ-driven design session)

> **Base codebase:** `WK.Axcess.ReturnDashboard` — React 18 + Vite client, Node 18 + Express + TypeScript server, Azure DevOps integration via PAT

1. Executive Summary

Three new capabilities are added to the existing **Regression Analyzer** module of BBT RADAR:

| # | Capability | UI Surface | Core Output |

|---|-----|-----|-----|

| **R1** | **Failure Prediction** | on Analyze Build | New **Prediction** column in the failed-tests table | Per-failure root-cause category + natural-language explanation + confidence |

| **R2** | **Environment Compare** | on Compare Builds | New **Env 1 vs Env 2** mode | Set-math diff (Both / Only Env1 / Only Env2) + LLM narrative summarizing what differs and why |

| **R3** | **Dev-PR Correlation** | New **Analyze with PR** input on Analyze Build & Compare Builds | Per failed test: `likelyCausedByPR`, confidence, explanation, suspect hunks |

All three features share a single **Prediction Service** (rules + AWS Bedrock LLM fallback) and a **Feedback Loop** that captures Approve / Reject / Edit user actions into a RAG memory used as few-shot exemplars on subsequent LLM calls.

2. Design Decisions (MCQ Outcomes)

| Q | Decision | Rationale |

|---|-----|-----|

| **Q1** — Prediction engine | **Hybrid: rules first, LLM fallback** | ~60–70 % of automation failures (timeout, element-not-found, assertion mismatch) classify deterministically from fields already on `FailedTest`. LLM only on the long tail → low cost, high accuracy, fully explainable. |

| **Q2** — LLM provider | **AWS Bedrock** (Claude 3.5 Sonnet primary, Llama 3 fallback) | Per directive. Bedrock SDK called server-side from Node/TS via IAM role. Cross-cloud egress documented (rest of stack runs in Azure). |

| **Q3** — Environment model | **Hybrid: environment registry + free-form override** | New `environments.json` config maps `Dev/QA/Stage/UAT/PreProd` → ADO definition IDs; user picks two from dropdowns, *or* pastes ad-hoc build URLs (today's flow preserved). |

| **Q4** — Diff semantics | **Full semantic LLM diff** (grounded on deterministic set-math) | Deterministic buckets feed the LLM as structured input; LLM returns narrative + categorized clusters. UI renders both. |

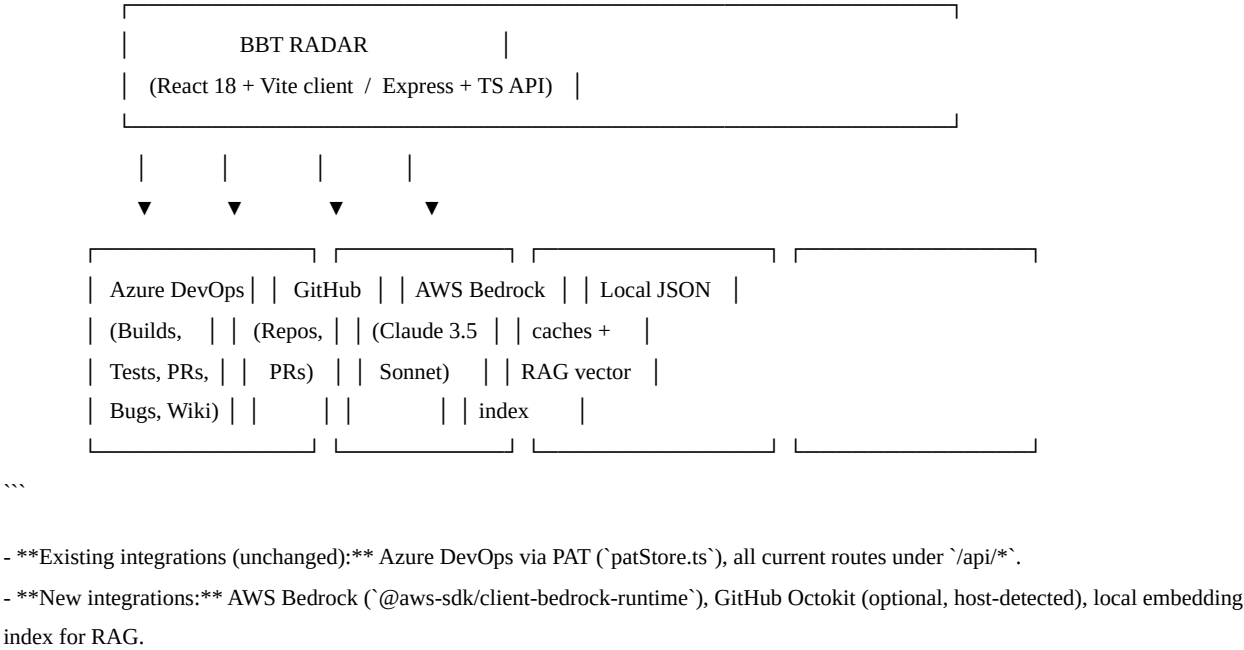
| **Q5** — PR source | **Both ADO Repos + GitHub, auto-detect by URL host** | Future-proofs against in-flight migration. `PrProvider` interface, two implementations. |

| **Q6** — PR correlation | **File-overlap + symbol/line proximity** → LLM root-cause | Rule pass narrows candidates; LLM ranks confidence per failure. Cheap & explainable. |

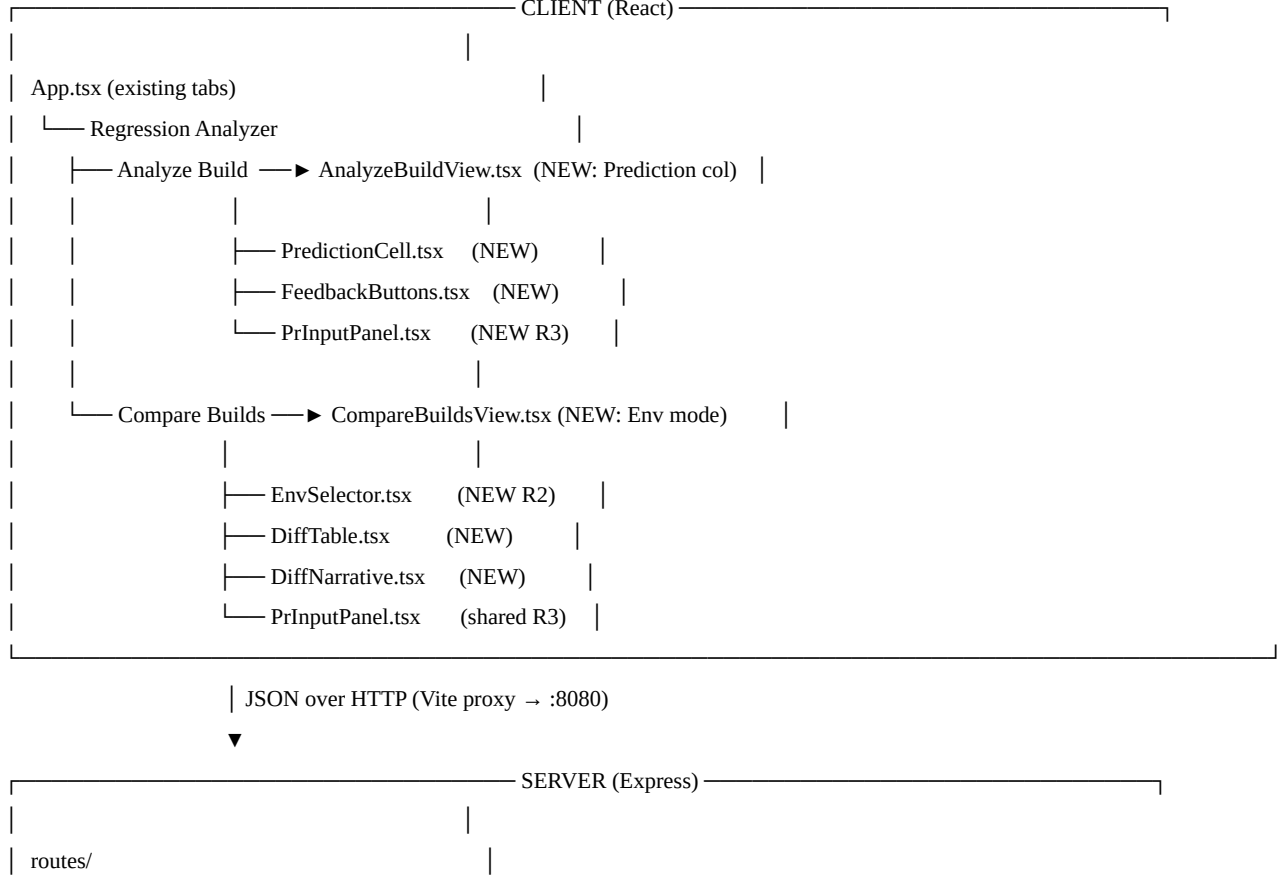
| **Q7** — Persistence | **Flat JSON files** under `dashboard/src/data/` | Consistent with `dashboard-cache.json`, `bugs-cache.json`, etc. No new infra. Keyed by stable SHA-256 hash; entries carry `cachedAt`. |

| **Q8** — Feedback loop | **Approve / Reject / Edit + RAG memory** | Edits become canonical cache entries *and* are embedded into a vector store of exemplars retrieved as few-shot context on future LLM calls. |

3. System Context



4. High-Level Architecture



	analyze.ts	(existing — extended to call PredictionService)	
	predict.ts	(NEW — explicit prediction endpoints)	
	envCompare.ts	(NEW — R2)	
	prCorrelate.ts	(NEW — R3)	
	feedback.ts	(NEW — Approve/Reject/Edit)	
	services/		
	PredictionService.ts	(NEW — orchestrator: rules → LLM → cache)	
	RuleClassifier.ts	(NEW — deterministic patterns)	
	BedrockClient.ts	(NEW — AWS SDK wrapper, retry, timeout)	
	PromptBuilder.ts	(NEW — assembles prompts + RAG exemplars)	
	EnvCompareService.ts	(NEW — set-math + LLM narrative)	
	PrCorrelationService.ts	(NEW — overlap + proximity + LLM)	
	FeedbackStore.ts	(NEW — append + canonicalize cache)	
	RagMemoryService.ts	(NEW — embed approved/edited predictions)	
	EnvironmentRegistry.ts	(NEW — loads environments.json)	
	providers/		
	PrProvider.ts	(NEW — interface)	
	AdoPrProvider.ts	(NEW — reuses adoClient + PAT)	
	GitHubPrProvider.ts	(NEW — Octokit)	
	existing (unchanged usage):		
	adoClient.ts, analyzer.ts, bugAnalyser.ts, patStore.ts		

...

5. Folder Structure (Enterprise Layout)

Additions are ****non-breaking****; existing files keep their paths.

...

```

dashboard/
├── src/
│   ├── client/
│   │   ├── App.tsx                (existing — tabs only)
│   │   ├── analyze/              (NEW directory)
│   │   │   ├── AnalyzeBuildView.tsx
│   │   │   ├── CompareBuildsView.tsx
│   │   │   ├── PredictionCell.tsx
│   │   │   ├── FeedbackButtons.tsx
│   │   │   ├── PrInputPanel.tsx
│   │   │   ├── EnvSelector.tsx
│   │   │   ├── DiffTable.tsx
│   │   │   └── DiffNarrative.tsx
│   │   ├── api/                  (NEW directory — typed clients)
│   │   └── predictionApi.ts

```

```

| | | └─ envCompareApi.ts
| | | └─ prCorrelateApi.ts
| | | └─ feedbackApi.ts
| | └─ types/ (NEW directory — barrel)
| | └─ prediction.ts
| | └─ envCompare.ts
| | └─ prCorrelation.ts
| └─ (all other existing files unchanged)
|
| └─ server/
|   └─ routes/
|     ├── analyze.ts (extended)
|     ├── predict.ts (NEW)
|     ├── envCompare.ts (NEW)
|     ├── prCorrelate.ts (NEW)
|     └─ feedback.ts (NEW)
|   └─ services/ (NEW directory)
|     ├── PredictionService.ts
|     ├── RuleClassifier.ts
|     ├── BedrockClient.ts
|     ├── PromptBuilder.ts
|     ├── EnvCompareService.ts
|     ├── PrCorrelationService.ts
|     ├── FeedbackStore.ts
|     ├── RagMemoryService.ts
|     └─ EnvironmentRegistry.ts
|   └─ providers/ (NEW directory)
|     ├── PrProvider.ts
|     ├── AdoPrProvider.ts
|     └─ GitHubPrProvider.ts
|   └─ (all other existing files unchanged)
|
| └─ data/
|   ├── environments.json (NEW — env registry)
|   ├── predictions-cache.json (NEW — keyed by sha256(testTitle|errSig|buildId))
|   ├── env-compare-cache.json (NEW — keyed by sha256(buildId1|buildId2))
|   ├── pr-correlation-cache.json (NEW — keyed by sha256(buildId|prUrl))
|   ├── prediction-feedback.json (NEW — append-only audit log)
|   ├── rag-exemplars.json (NEW — {id, text, embedding[], canonicalAnswer})
|   └─ rule-patterns.json (NEW — editable rule library)
|
| └─ docs/
|   └─ architecture/
|     └─ PREDICTION-AND-PR-CORRELATION.md (this document)

```

...

6. Domain Model Extensions

Added to `dashboard/src/client/types.ts` (and a server-side mirror):

```
``ts
// R1 — Prediction
export interface FailurePrediction {
  category: 'Timeout' | 'ElementNotFound' | 'AssertionMismatch' | 'NetworkError'
    | 'EnvironmentDown' | 'Flaky' | 'DataDependency' | 'PrRelated' | 'Unknown';
  reason: string;           // 1–2 sentence natural-language explanation
  confidence: number;       // 0..1
  source: 'rule' | 'llm' | 'cache' | 'feedback-canonical';
  ruleId?: string;         // when source = 'rule'
  modelId?: string;        // when source = 'llm' (e.g., 'anthropic.claude-3-5-sonnet')
  exemplarIds?: string[];  // RAG few-shot ids used
  predictedAt: string;      // ISO
}

// FailedTest gains a prediction
export interface FailedTest {
  /* existing fields ... */
  prediction?: FailurePrediction;
}

// R2 — Environment Compare
export interface EnvironmentRef {
  envName: string;          // 'QA' | 'Stage' | ... or 'Custom'
  buildId: string;
  buildUrl: string;
  buildNumber: string;
}

export interface EnvDiffBuckets {
  failingInBoth: TestPairDiff[]; // same title, may differ in errorSignature
  onlyInEnv1: FailedTest[];
  onlyInEnv2: FailedTest[];
  passingInBoth: number;        // count only
}

export interface TestPairDiff {
  testCaseTitle: string;
  env1: FailedTest;
  env2: FailedTest;
  errorSignatureChanged: boolean;
}

export interface EnvCompareResult {
  env1: EnvironmentRef;
  env2: EnvironmentRef;
  buckets: EnvDiffBuckets;
  narrative: {                // LLM output
```

```

summary: string;
env1RootCauses: { category: string; count: number; note: string }[];
env2RootCauses: { category: string; count: number; note: string }[];
keyDifferences: string[];
};
generatedAt: string;
}

// R3 — PR Correlation
export interface PrRef {
  provider: 'ado' | 'github';
  url: string;
  id: string;
  title: string;
  author: string;
  sourceBranch: string;
  targetBranch: string;
}

export interface PrChange {
  filePath: string;
  changeType: 'add' | 'edit' | 'delete' | 'rename';
  hunks: { startLine: number; endLine: number; addedSymbols: string[]; removedSymbols: string[] }[];
}

export interface PrCorrelation {
  failedTest: FailedTest;
  likelyCausedByPR: boolean;
  confidence: number; // 0..1
  explanation: string; // LLM natural language
  suspectHunks: { filePath: string; startLine: number; endLine: number }[];
  matchedBy: ('file-path' | 'symbol' | 'stack-frame')[];
}

export interface PrCorrelationResult {
  pr: PrRef;
  buildId: string;
  correlations: PrCorrelation[];
  generatedAt: string;
}
...

---
```

7. API Endpoints

All endpoints sit on the existing Express server, share the existing PAT resolution, and return JSON. Errors follow the existing `{ error: string }` shape.

7.1 Prediction

Method	Path	Body	Response	Purpose
-----	-----	-----	-----	-----

| `POST` | `/api/predict/failures` | `{ buildUrl, pat?, useCache?: true, forceLLM?: false }` | `{ buildId, predictions: FailurePrediction[] }`
(one per failed test, indexed by `testCaseTitle`) | Run prediction pipeline on every failure of a single build. |
| `POST` | `/api/predict/failure` | `{ failedTest: FailedTest, contextBuildId?, forceLLM?: false }` | `FailurePrediction` | Predict a single
failure (used by client for on-demand cell refresh). |
| `GET` | `/api/predict/cache/:buildId` | — | `FailurePrediction[]` | Read cached predictions for a build. |
| `DELETE` | `/api/predict/cache/:buildId` | — | `{ cleared: number }` | Invalidate cache entries (admin/debug). |

****Existing endpoint**** `POST /api/analyze` (dashboard/src/server/routes/analyze.ts) is extended to
****optionally**** invoke predictions when query param `withPrediction=true` is supplied, so legacy callers are unaffected.

7.2 Environment Compare

Method	Path	Body	Response	Purpose
`GET`	`/api/environments`	—	`{ environments: { name, definitionId, baseUrl, description }[] }`	List the registry from `environments.json`.
`POST`	`/api/env-compare`	`{ env1: { envName?, buildUrl? }, env2: { envName?, buildUrl? }, pat?, withNarrative?: true }`	`EnvCompareResult`	Compute deterministic diff buckets + LLM narrative. Resolves `envName` → latest build via registry, else uses `buildUrl`.
`GET`	`/api/env-compare/cache/:key`	—	`EnvCompareResult`	Retrieve a prior comparison.

7.3 PR Correlation

Method	Path	Body	Response	Purpose
`POST`	`/api/pr/resolve`	`{ prUrl, pat?, githubToken? }` `{ pr: PrRef, changes: PrChange[] }`	—	Auto-detect host, fetch PR metadata + unified diff.
`POST`	`/api/pr/correlate`	`{ buildUrl, prUrl, pat?, githubToken?, useCache?: true }`	`PrCorrelationResult`	Correlate failed tests of a build with a PR.
`GET`	`/api/pr/correlate/cache/:key`	—	`PrCorrelationResult`	Cached result.

7.4 Feedback

Method	Path	Body	Response	Purpose
`POST`	`/api/feedback/prediction`	`{ predictionKey, action: 'approve' \  'reject' \  'edit', editedReason?, editedCategory?, userId }`	—	Capture user decision. On `edit` / `approve` the cache entry is marked canonical and a RAG exemplar is queued for embedding.
`GET`	`/api/feedback/stats`	`?since=ISO`	`{ total, approved, rejected, edited, accuracyByCategory: { ... } }`	Quality dashboard.
`POST`	`/api/feedback/rag/rebuild`	—	`{ exemplars: number }`	Re-embed all canonical entries (admin).

7.5 Existing & shared

Method	Path	Purpose
`GET`	`/api/health`	unchanged
`GET`	`/api/pat/status`	unchanged (patStore.ts)
`POST`	`/api/pat/reload`	unchanged (patStore.ts)

8. Data Flows

8.1 R1 — Analyze Build with Prediction column

...

User clicks "Analyze" on a build URL

|

▼

Client: POST /api/analyze?withPrediction=true

|

Server: analyze.ts

└─ analyzeBuild() — existing (adoClient.ts) → AnalysisResult

└─ for each failedTest:

 PredictionService.predict(failedTest, buildId)

 └─ 1. CacheLookup (predictions-cache.json, key=sha256(title|errSig|buildDefId))

 └─ HIT → return {source:'cache' or 'feedback-canonical'}

 └─ 2. RuleClassifier.classify(failedTest)

 └─ matched rule with confidence ≥ 0.85 → return {source:'rule'}

 └─ 3. RagMemoryService.retrieveTopK(failedTest, k=3) → exemplars

 └─ 4. PromptBuilder.build(failedTest, exemplars, codeContext?)

 └─ 5. BedrockClient.invoke(prompt) → JSON {category, reason, confidence}

 └─ 6. CacheWrite + return {source:'llm'}

|

▼

Response: AnalysisResult with `failedTests[i].prediction` populated

|

Client renders new "Prediction" column with:

- category chip (color-coded)
- reason text (truncated, expand-on-click)
- confidence bar
- Approve / Reject / Edit buttons (FeedbackButtons.tsx)

...

8.2 R2 — Compare Builds across two environments

...

User picks Env1=QA, Env2=Stage from EnvSelector (or pastes two build URLs)

|

Client: POST /api/env-compare

|

Server: envCompare.ts

└─ EnvironmentRegistry.resolveLatestBuild(envName) // when envName given

└─ parallel: analyzeBuild(env1) + analyzeBuild(env2) // existing

└─ parallel: PredictionService.predictBatch(env1.failed) + (env2.failed)

└─ EnvCompareService.computeBuckets(env1, env2)

 • set math on testCaseTitle

 • for "failing in both", compare normalized errorSignature

 (errorMessage stripped of GUIDs, paths, line numbers)

└─ if (withNarrative):

 PromptBuilder.envDiffPrompt(buckets, predictionsEnv1, predictionsEnv2)

 BedrockClient.invoke() → {summary, env1RootCauses[], env2RootCauses[], keyDifferences[]}

|



Response: EnvCompareResult

|

Client renders:

- DiffTable.tsx — 4 sections (Both / Only-Env1 / Only-Env2 / Passing-both count)
- DiffNarrative.tsx — markdown summary + categorized cluster cards
- Each row carries the Prediction + Feedback controls (reuses R1 components)

...

8.3 R3 — Dev PR correlation

...

User pastes PR URL into PrInputPanel and clicks "Correlate"

|

Client: POST /api/pr/correlate { buildUrl, prUrl }

|

Server: prCorrelate.ts

```
|— provider = host==='github.com' ? GitHubPrProvider : AdoPrProvider
|— provider.fetchPr(prUrl)      → PrRef
|— provider.fetchChanges(prUrl) → PrChange[] (unified diff, hunks, symbols)
|— analyzeBuild(buildUrl)      → failedTests[]
|
|— for each failedTest:
  PrCorrelationService.correlate(failedTest, changes):
    |— 1. RULE PASS (overlap + proximity):
    |   • filePathMatch = changes.any(c => normalize(c.filePath) === failedTest.sourceFile)
    |   • symbolMatch   = changes.any(c => c.hunks.flatMap(h=>h.addedSymbolsUremovedSymbols)
    |                                     .includes(failedTest.methodName))
    |   • stackFrameMatch = parse failedTest.errorMessage for "at X.Y(file:line)";
    |                     intersect with hunk ranges
    |   suspectHunks = union of matches
    |   if (no match) → emit {likelyCausedByPR:false, confidence:0, explanation:'No overlap', matchedBy:[]}
    |
    |— 2. LLM PASS (only when suspectHunks not empty):
    |   PromptBuilder.prCorrelationPrompt(failedTest, pr, suspectHunks)
    |   BedrockClient.invoke() →
    |       {likelyCausedByPR:boolean, confidence:0..1, explanation:string}
```

|

Server writes pr-correlation-cache.json

|

Client renders correlation panel beneath each failed test:

- verdict badge (Likely caused by PR / Unrelated / Inconclusive)
- suspect hunks (file:line) with deep-link to PR file diff
- LLM explanation + Approve/Reject/Edit

...

8.4 Feedback → RAG memory loop

...

User clicks Approve / Reject / Edit on a prediction

|

Client: POST /api/feedback/prediction

|

Server: feedback.ts

|— FeedbackStore.append(entry) → prediction-feedback.json (audit log)

|— if (action==='approve' || action==='edit'):

| predictions-cache.json[key] = { ...canonical, source:'feedback-canonical' }

| RagMemoryService.enqueueEmbed(entry)

|

|— (async) RagMemoryService.flush():

 for each queued entry:

 embedding = BedrockClient.embed(entry.text) // titan-embed-text-v2

 append to rag-exemplars.json

 build/refresh in-memory cosine-similarity index

|

On the next PredictionService.predict() call:

 RagMemoryService.retrieveTopK(failedTest, k=3)

→ top-3 approved/edited exemplars are inserted into the LLM prompt

as few-shot examples → measurably reduces hallucination over time.

...

9. Rule Library (deterministic layer)

`rule-patterns.json` is a hot-editable list of rules; the server reloads on file change (similar to `patStore.ts` reload pattern). Each rule:

```jsonc

{

  "id": "rule.timeout.selenium",

  "category": "Timeout",

  "match": {

    "errorMessage": "/TimeoutException|wait timed out|element click intercepted/i",

    "durationTrend": "increasing",

    "minDurationMs": 30000

  },

  "reasonTemplate": "Selenium wait timed out after {{durationInMs}}ms on '{{methodName}}'. Element may be slow to render or selector outdated.",

  "confidence": 0.9

}

```

Initial rule set (seed): `Timeout`, `ElementNotFound`, `AssertionMismatch`, `NetworkError` (HTTP 5xx / connection-reset),

`EnvironmentDown` (>50 % of suite failed with same network error), `Flaky` (passed on retry — uses `attempts[]` already on `FailedTest`).

10. LLM Prompt Strategy

`PromptBuilder.ts` produces strictly-typed JSON-mode prompts (Bedrock Claude `tool\_use` or response-prefix `{` trick). Common envelope:

...

SYSTEM:

You are a senior SDET analyzing automation failures for the Wolters Kluwer BBT (Browser Based Tax) regression suite. Respond ONLY with valid JSON matching the schema. Be concise (≤ 2 sentences for "reason"/"explanation").

EXEMPLARS (RAG top-K, source=feedback-canonical):

{{#each exemplars}}

- Input: { test: "...", error: "...", code: "..." }

Output: { category: "...", reason: "..." }

{{/each}}

INPUT:

{{taskSpecificContext}}

OUTPUT SCHEMA:

{{schema}}

...

Three prompt templates:

1. `**predictionPrompt**` — input = single failed test + code context + exemplars.
2. `**envDiffPrompt**` — input = bucket counts + top-N failures per env with their predictions.
3. `**prCorrelationPrompt**` — input = single failed test + PR title/description + suspect hunks (truncated to 200 lines).

Token guards: every prompt is hard-capped (e.g., 8K input tokens) by truncating `failingCode`, `errorMessage`, and hunks with ellipsis + line-count footer.

11. Security & Governance

| Area | Approach |

|-----|-----|

| **ADO PAT** | Unchanged — [patStore.ts](dashboard/src/server/patStore.ts); never sent to client; never logged. |

| **GitHub token** | Resolved from env `GITHUB\_TOKEN` server-side only. Optional per-request override in body (mirrors PAT pattern). |

|

| **AWS Bedrock auth** | IAM role attached to the container (preferred) or env-var keys (`AWS\_ACCESS\_KEY\_ID`, `AWS\_SECRET\_ACCESS\_KEY`, `AWS\_REGION`). No client exposure. |

| **PII / customer data** | Failure messages may contain return-fragment text. `PromptBuilder` runs a **redactor** that strips SSN-shaped, EIN-shaped, and email patterns before LLM submission. Audit log captures pre-redaction hash only. |

| **Cross-cloud egress** | Azure-hosted container → AWS Bedrock endpoint over TLS. Document in network review; allow-list `bedrock-runtime.<region>.amazonaws.com`. |

| **Prompt-injection defense** | LLM responses are JSON-schema-validated server-side; any free-form text from LLM is rendered as plain text in React (no `dangerouslySetInnerHTML`). |

| **Rate limiting** | Per-IP token bucket on `/api/predict/` and `/api/pr/correlate` to prevent runaway Bedrock cost. |

| **OWASP** | Existing redaction in `adoClient.ts` extended to cover AWS strings. Input validation on all new route bodies via a thin zod schema. |

12. Caching & Performance

| Layer | TTL / Invalidation |

|-----|-----|

| `predictions-cache.json` | Keyed by `sha256(testCaseTitle \ normalizedErrorSig \ buildDefinitionId)`. **No TTL** — entries are immutable per (test, error-signature) until user marks canonical via feedback. |

| `env-compare-cache.json` | Keyed by `sha256(buildId \ buildId2 \ narrativeFlag)`. TTL = 7 days. |

| `pr-correlation-cache.json` | Keyed by `sha256(buildId \ prUrl \ prIterationId)`. Invalidated when PR has new iteration. |

| `rag-exemplars.json` | Loaded into memory at boot; refreshed on every approve/edit. |

| In-memory | Each cache file kept in a `Map<string, T>` on server boot; writes are append-then-fsync. |

Concurrency on JSON writes uses a simple in-process mutex (same pattern as existing bug cache). Single-replica Docker deployment makes this safe today; multi-replica is called out as a future migration to SQLite.

13. Deployment & Config

No change to the existing Azure Pipelines / Docker / Azure Storage flow. New environment variables:

| Var | Purpose | Default |

|----|-----|-----|

| `BBT\_PAT\_FILE` | (existing) ADO PAT file path | `src/data/pat.json` |

| `BBT\_BEDROCK\_REGION` | AWS region for Bedrock | `us-east-1` |

| `BBT\_BEDROCK\_MODEL\_ID` | Primary model | `anthropic.claude-3-5-sonnet-20241022-v2:0` |

| `BBT\_BEDROCK\_FALLBACK\_MODEL\_ID` | Fallback when primary throttled | `meta.llama3-70b-instruct-v1:0` |

| `BBT\_BEDROCK\_EMBED\_MODEL\_ID` | Embeddings for RAG | `amazon.titan-embed-text-v2:0` |

| `AWS\_ACCESS\_KEY\_ID` / `AWS\_SECRET\_ACCESS\_KEY` | When IAM role unavailable | — |

| `GITHUB\_TOKEN` | GitHub PR fetch | (optional) |

| `BBT\_LLM\_DAILY\_BUDGET\_USD` | Hard cap; pipeline rejects calls past budget | `50` |

`environments.json` shape:

```jsonc

```
{
 "environments": [
 { "name": "QA", "definitionId": 1234, "baseUrl": "https://wkaccess.visualstudio.com/...", "description": "Nightly QA regression" },
 { "name": "Stage", "definitionId": 1245, "baseUrl": "https://wkaccess.visualstudio.com/...", "description": "Pre-prod" }
]
}
```

---

## ## 14. Output / Export Formats

The new views reuse the existing XLSX export utilities pattern ([bugExcelExport.ts](dashboard/src/client/bugExcelExport.ts), [signoffResultsExport.ts](dashboard/src/client/signoffResultsExport.ts)).

| View | Export buttons | Sheets / Files |

|-----|-----|-----|

| Analyze Build (R1) | `xlsx`, `csv`, `json` | `Failures` sheet with extra columns: *Prediction Category*, *Prediction Reason*, *Confidence*, *Source*, *Feedback* |

| Compare Builds (R2) | `.xlsx`, `.json` | Sheets: `Summary`, `Failing-in-Both`, `Only-Env1`, `Only-Env2`, `Narrative` |  
| PR Correlation (R3) | `.xlsx`, `.json` | Sheet `PR-Correlation` with: `\*Test\*`, `\*Verdict\*`, `\*Confidence\*`, `\*Suspect File:Line\*`, `\*Explanation\*` |

---

## ## 15. Feedback, Observability & Quality Metrics

- `**prediction-feedback.json**` is an append-only audit log of every Approve/Reject/Edit with `userId`, timestamp, previous prediction, new prediction.
- `**/api/feedback/stats**` computes rolling accuracy:  
`accuracy = approved / (approved + rejected + edited)` overall and per category.
- A small `**Quality**` page (under the Regression Analyzer parent tab) renders this with Recharts (already a dependency).
- Server logs every LLM call with `{modelId, promptTokens, completionTokens, latencyMs, cacheHit}` for cost/perf monitoring.

---

## ## 16. Risks & Mitigations

| Risk | Mitigation |  
|-----|-----|  
| LLM hallucination on novel failures | Strict JSON schema + low temperature (0.2) + RAG exemplars + confidence threshold; below 0.5 renders as `*Inconclusive*`. |  
| Bedrock cost spike | Hybrid rule-first; per-day budget env var; cache-first lookups; PR pass uses LLM only when overlap exists. |  
| Cross-cloud latency (Azure → AWS) | Bedrock region pinned to lowest-latency US region; client shows skeleton + streams predictions row-by-row. |  
| Flat-file write contention | Single-replica today + in-process mutex; migration path to SQLite documented in Q7. |  
| PII leakage to LLM | Redactor in `PromptBuilder`; pre-redaction text never leaves server unhashed. |  
| PR diff too large | Hard cap 200 lines per hunk + drop hunks unrelated to suspect files. |

---

## ## 17. Phased Delivery

| Phase | Scope | Exit criteria |  
|-----|-----|-----|  
| `**P1**` | RuleClassifier + Prediction column (no LLM, no feedback) | Analyze Build shows category + reason for ≥60 % of failures. |  
| `**P2**` | Bedrock integration + LLM fallback + cache | Predictions for remaining failures; cache hit ≥80 % on re-runs. |  
| `**P3**` | Environment registry + Compare Builds two-env mode + LLM narrative | QA-vs-Stage comparison demo with narrative. |  
| `**P4**` | PR correlation (ADO provider first, GitHub second) | At least one real PR correctly attributed to a failure. |  
| `**P5**` | Feedback (Approve/Reject/Edit) + RAG memory | Stats dashboard live; canonical entries override predictions. |  
| `**P6**` | Excel/CSV/JSON exports for all three views + Quality page | UAT sign-off. |

---

## ## 18. Out of Scope (v1)

- Multi-tenant separation (single WK Axxess tenant assumed).
- SaaS-style billing / quotas per user.
- Auto-creation of ADO bugs from predictions (future: link the `*Approve*` action to ``bugAnalyser`` to spawn a Bug work item).
- Multi-replica scale-out (SQLite migration is a P7+ item).
- Confluence / Jira / Browserstack ingestion (those were in your generic brief but are `*not*` required for R1/R2/R3 and are deferred).

---

## ## 19. Appendix — Endpoint Quick Reference

---

### # Prediction

POST /api/predict/failures

POST /api/predict/failure

GET /api/predict/cache/:buildId

DELETE /api/predict/cache/:buildId

### # Environment Compare

GET /api/environments

POST /api/env-compare

GET /api/env-compare/cache/:key

### # PR Correlation

POST /api/pr/resolve

POST /api/pr/correlate

GET /api/pr/correlate/cache/:key

### # Feedback / RAG

POST /api/feedback/prediction

GET /api/feedback/stats

POST /api/feedback/rag/rebuild

### # Existing (unchanged)

POST /api/analyze (extended: ?withPrediction=true)

GET /api/health

GET /api/pat/status

POST /api/pat/reload

---

---

\*End of architecture document.\*